

9.3 Ollama

Like GPT4All, Ollama is a tool that allows you to run AI models locally on your system. In contrast to GPT4All, however, Ollama doesn't provide a GUI, but focuses on local execution and the management of models in the terminal. The application is so flexible that you can add a graphical interface or integrations into other applications on your system in just a few steps.

Ollama is an open-source project managed on GitHub (<https://github.com/ollama/ollama>). The official website of the project with admittedly somewhat sparse documentation, but an interesting blog can be found at <https://ollama.com>.

Ollama is able to make optimal use of your system's hardware and can use both the system's CPU and GPU to run the models. You should give preference to using the GPU, as the performance is significantly better. The platform supports a range of NVIDIA and AMD graphics cards as well as GPU acceleration on Apple devices. An overview of the supported graphics cards can be found at <https://github.com/ollama/ollama/blob/main/docs/gpu.md>.

9.3.1 Benefits of Ollama

Ollama is a lightweight and flexible platform that allows you to run your local models. There are several reasons why you should use this platform:

- **Local execution**

With Ollama, as with GPT4All, you're independent of AI solution providers, as all elements of the platform, including the models, run locally on your system. In addition to this independence, local execution also reduces communication latency so that you receive your answers faster.

- **Simple operation**

Using the Ollama command-line tool, you can carry out all the necessary operations directly with a straightforward set of commands.

Although this variant isn't particularly convenient, it allows you to run Ollama on a system without a GUI and control it via a remote connection.

- **Preconfigured models**

Ollama provides you with a range of predefined models that you can load directly via the command line and integrate into your system.

- **Customizability**

Loading the predefined models is the most convenient way to get started quickly with frequently used, freely available models. However, Ollama also allows you to integrate your own models.

- **Flexibility**

In the Ollama tool, freedom and flexibility run through everything from the models to the UIs. You can design the platform entirely according to your needs with hardly any restrictions.

- **Privacy**

Local execution means that your prompts and responses don't leave your system. If you run Ollama locally, your data is protected against unauthorized access. If you operate the platform in your local network or even publicly available on the internet, you must protect the interfaces against misuse.

- **Cross-platform**

You can run Ollama on different systems. There are installation routines for macOS, Linux, and Windows. You can also operate the platform in a Docker container.

Now that you know the main advantages of Ollama, let's take a closer look at the general structure of the platform and its individual components.

Understanding how Ollama is structured will help you better understand how it works and the customization options that are available.

9.3.2 General Structure of Ollama

Ollama tries to make it as easy as possible to start working with local models, so that you have a working environment in just a few steps. Ollama

consists of several parts that work together to create a functioning local environment:

- **Ollama engine**

The core of Ollama is the engine that is responsible for running the AI models. It allocates resources and optimizes execution. The engine runs as a service in the background and provides the interfaces to the model.

- **Command-line tool**

The easiest way to interact with Ollama is via the command line. This tool allows you to manage the local models and also send prompts.

- **Local model**

The Ollama platform alone is of little use without at least one local model installed. You can use the command line to load one of the models from the Ollama library or integrate your own model.

You must install these components on your system in the next step to be able to use Ollama.

9.3.3 Local Installation and Execution

There are various options available for installing Ollama. The first port of call is the official website (<https://ollama.com>) where you get the Ollama application for macOS, which you can run immediately. An installation script is provided for Linux, which you can download via an HTTP client such as curl and pipe into your shell. A setup file is available for download to Windows.

Alternatively, you can also use your system's package manager. On macOS, you can install Ollama with Homebrew, and on an Ubuntu system, you can install it using snap.

After the installation, a range of command-line options are available to you. You can test the general functionality on the command line via the following command:

```
ollama -v
ollama version is 0.3.6
```

Before you can work with Ollama, you must install a local model. The following models are available in the Ollama library:

- **Llama 3.1**

A powerful language model from Meta designed specifically for NLP and conversational AI. It provides improvements in text generation and text comprehension compared to its predecessors.

- **Phi 3**

An open LLM from Microsoft that delivers very good performance with a relatively small size and performs excellently in various benchmarks in language processing, logic, programming, and mathematics.

- **Gemma 2**

A generative model from Google that is characterized by high efficiency, performance, and accessibility. It was developed primarily for developers and researchers.

- **Llama 2 Uncensored**

A variant of Llama 2 that is less restrictive in terms of content moderation and can therefore be used more flexibly in content generation, especially in contexts where less controlled text output is required.

- **Mistral**

A model that specializes in the recognition and processing of multilingualism. It's designed for applications that have to work in several languages and require a high degree of accuracy in language processing.

- **Starling**

A model that specializes in precision in the reproduction of information. It's often used in applications that require reliable and accurate information, such as question-and-answer systems or knowledge databases.

- **Code Llama**

A specialized model from Meta for code generation and completion. It supports multiple programming languages and is particularly useful for developers looking for support with code creation and optimization.

- **LLaVA**

A model that combines multimodal capabilities, such as the processing of text and visual data. It's ideal for applications that require the integration of image and text information such as image descriptions or visual question-and-answer systems.

For some models, such as Llama 3.1, there are different variants available. These range from the smallest version with 8 billion parameters and 4.7 GB to the medium version with 70 billion parameters and 40 GB to the largest version with 405 billion parameters and 231 GB. If there are several variants, which is reflected in the name. For example, the large Llama 3.1 model is called `llama3.1:405b`. For the first steps, we install the 8B version of Llama 3.1 by using the following command:

```
ollama pull llama3.1

pulling manifest
pulling 8eeb52dfb3bb... 100% 4.7 GB
pulling 11ce4ee3e170... 100% 1.7 KB
pulling 0ba8f0e314b4... 100% 12 KB
pulling 56bb8bd477a5... 100% 96 B
pulling 1a4c3c319823... 100% 485 B
verifying sha256 digest
writing manifest
removing any unused layers
success
```

By using `ollama list`, you can display the locally available models. You'll receive information such as the name of the model, its unique ID, the size, and the date of the last change. Depending on the system, Ollama saves the models either in a subdirectory of the current user (macOS and Windows) or in a subdirectory of `/usr/share` (Linux).

The `ollama serve` command starts the Ollama service, and you can run a specific model via `ollama run`. By using `ollama run llama3.1`, you start an interactive session on the command line in which you can send any prompts, such as the following:

Prompt: Create a TypeScript function that adds two numbers.

Here is a simple TypeScript function that takes two numbers as parameters and returns their sum:

```
/**
 * Adds two numbers together.
```

```
*
* @param num1 The first number to add.
* @param num2 The second number to add.
* @returns The sum of the two numbers.
*/
function addNumbers(num1: number, num2: number):
number {
    return num1 + num2;
}

// Example usage:
console.log(addNumbers(5, 7)); // Outputs: 12
```

[explanation of the code]

The result of the prompts is impressive, even with the small Llama 3.1 model. The larger the model, the better the feedback. You also have the option of getting even more out of your LLM through fine-tuning or retrieval-augmented generation (RAG). You can find out more about this topic in [Chapter 12](#).

As the models are sometimes quite memory-intensive, there will come a time when you want to remove a locally installed model. In this case, you should rely on the command-line tool and not attempt to delete the model files from the hard disk yourself. For example, to remove the Llama 3.1 model, you can use the `ollama rm llama3.1` command. This command ensures that the model and all associated metadata gets properly deleted.

9.3.4 Running Ollama in a Docker Container

The local installation of Ollama has a decisive disadvantage: it interferes with your system. While Ollama is a pretty solid tool to work with, if you install every new AI tool on your system, it can quickly become confusing. An alternative to this is to run Ollama in a container. This means you remain independent of your local system and can even use the container on other systems, be it on other client systems or servers.

To use the Ollama container, you need the Docker engine or an alternative such as Rancher Desktop on your system. If this requirement is met, you

can use the official Ollama image:

```
docker run -d -v ollama:/root/.ollama -p 11434:11434 \
  --name ollama ollama/ollama
```

This command starts a container in the background (`-d`), which is based on the official Ollama image (`ollama/ollama`). The `ollama` volume is mounted to the `/root/.ollama` path in the container. By default, the Ollama API is linked to port 11434. The `-p` option allows you to release this port to the outside world so that you can access it from your system. You also assign the `ollama` name to the container to make it easier to address.

Depending on whether you have a GPU from NVIDIA or AMD in your system, you'll need to take additional steps to enable support for your GPU. You can find detailed instructions at <https://hub.docker.com/r/ollama/ollama>.

When you run the command, Docker first searches for a local version of the image. If that isn't available, Docker downloads the image from the central registry and creates and starts the container. As soon as the container has been started, you can interact with it.

Usually, you use the `docker exec -it ollama` command followed by the actual Ollama command. This Docker command executes the specified command in the running container named `ollama`. The `i` option stands for an interactive session, and `t` stands for TTY, a pseudo-terminal. Using `docker exec -it ollama I run llama3.1`, you start an interactive command-line session with the Llama 3.1 model. Via the locally mounted volume, Ollama downloads the models to the Ollama directory of your host system so that the data is independent of the container.